VIVA VOCE CRITICAL DOSSIER • FINAL YEAR BSC IT EXAMINATION

CyberShield & Codexa Defense Viva Preparation

CANDIDATE: **Codewithjainam7 (Jainam)** DOMAINS: **Full Codexa (Front & Back) + CyberShield Frontend** SYSTEM VER: **Production v9.4**

CORE CODEXA ARCHITECTURE #01

Abstract Syntax Tree (AST) & Binary Parse Trees

In **Codexa**, we don't treat source code as raw text strings. When a user submits code for AI security analysis, it is parsed into an **Abstract Syntax Tree (AST)**.

VIVA SUMMARY (Hinglish):
Compiler ya interpreter jab code read karta hai, toh pehle *Lexical Analysis* se code ko **Tokens** mein todta hai (keywords, variables, operators). Phir *Syntax Analysis* in tokens ko ek hierarchical tree mein arrange karta hai jise **Abstract Syntax Tree (AST)** kehte hain.

- **Node Structure:** Each node represents a syntactic construct (e.g. `FunctionDeclaration`, `BinaryExpression`, `IfStatement`, `VariableDeclarator`).
- **Binary Expression Tree:** An arithmetic or logical operation like `a + b * c` is represented as a Binary Tree where the operator is the root and operands are the left and right children.
- **Why Codexa Uses AST:**
  1. Detects insecure function calls (e.g., `eval()`, `exec()`, `dangerouslySetInnerHTML`) by walking the AST nodes.
  2. Accurately localizes bugs and line numbers without regex guessing.
  3. Transforms AST nodes to generate automated zero-shot vulnerability patches!

EXAMINER QUESTION: "What is an AST and why did you need it in Codexa instead of just sending code to ChatGPT?"

"Sir/Ma'am, sending raw code as a string to an LLM is error-prone. In Codexa, we parse the code into an Abstract Syntax Tree to break down the syntax into exact tokens and parent-child scopes. This allows us to perform deterministic static code analysis, identify exact vulnerable nodes (like SQL query concatenations or unsafe memory buffers), and pass enriched contextual metadata to the LLM for guaranteed zero-shot security patches."

SECURITY STANDARD #02

OWASP Top 10 Vulnerabilities Explained

OWASP (Open Web Application Security Project) publishes the industry-standard ranking of the most critical security risks facing web applications.

VIVA SUMMARY (Hinglish):
OWASP web security ka global gold standard hai. Humne CyberShield ke *Developer Footprint Module* aur *Site Security Scanner* mein OWASP Top 10 baselines ko integrate kiya hai live header aur code checks ke liye.

- **A01: Broken Access Control:** Users accessing resources or actions outside their permissions (e.g. IDOR - changing user ID in URL to view another profile). *Fix: Server-side RBAC token verification.*
- **A02: Cryptographic Failures:** Sensitive data transmitted in plaintext or hashed with broken algorithms (MD5/SHA1). *Fix: TLS 1.3, AES-GCM-256, bcrypt/Argon2.*
- **A03: Injection (SQLi, Command Injection, XSS):** Untrusted input parsed directly by an interpreter.
  - *SQL Injection:* `' OR '1'='1` bypasses login. Fix: Parameterized queries.
  - *Cross-Site Scripting (XSS):* Attacker injects malicious JS into web page. Fix: DOMPurify & CSP.
- **A05: Security Misconfiguration:** Leaving debug mode enabled, default credentials, or missing security headers (`Content-Security-Policy` , `Strict-Transport-Security`).
- **A08: Software & Data Integrity Failures:** Loading unverified third-party libraries without Subresource Integrity (SRI) hashes.

EXAMINER QUESTION: "Where did you implement OWASP in your project?"

"In CyberShield, our Site Scanner automatically checks target websites against OWASP A05 (Security Misconfigurations) by validating HSTS, CSP, and X-Frame-Options headers. In Codexa, our static analysis scanner inspects code for OWASP A03 Injection flaws and automatically generates parameterized SQL replacements."

PRIVACY-PRESERVING CRYPTOGRAPHY #03

k-Anonymity Model in Password & Identity Auditing

In CyberShield's Compromised Identity Auditor, how do we search 10 billion leaked passwords without compromising the user's password? Answer: **The k-Anonymity Mathematical Model.**

VIVA SUMMARY (Hinglish):
Agar user apna password check kar raha hai, toh agar hum uska password server pe bhejenge toh wo khud ek security leak ban jayega! Isliye hum **k-Anonymity** use karte hain: password ka pehle SHA-1 hash nikaalte hain client-side par, aur sirf pehle **5 characters** server ko bhejte hain. Server hazaaron hashes ka matching group return karta hai, aur baki match user ke browser mein hota hai!

- **Mathematical Definition:** A dataset has *k-anonymity* if information for each person cannot be distinguished from at least `k - 1` other individuals whose information also appears in the release.
- **Step-by-Step Execution in CyberShield:**
  1. User enters password: `P@ssw0rd123`.
  2. Browser computes SHA-1: `5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8`.
  3. Client splits hash: **Prefix** (first 5 chars: `5BAA6`) and **Suffix** (remaining 35 chars: `1E4C9B...`).
  4. Client sends ONLY `5BAA6` to HaveIBeenPwned range API.
  5. API returns ~500 hash suffixes sharing that prefix (k > 500).
  6. Client searches for its suffix locally in RAM. If found, shows leak count.

EXAMINER QUESTION: "If I enter my bank password into CyberShield, how do I know you won't steal it?"

"Sir/Ma'am, CyberShield mathematically cannot steal your password because of the k-Anonymity protocol. Your password never leaves your browser. We compute the SHA-1 hash locally and transmit only a 5-character prefix. The server responds with hundreds of candidate suffixes, and the final verification happens strictly in your browser's private memory."

MACHINE LEARNING ALGORITHM #04

Random Forest Classifier in Phishing & Heuristics

Random Forest is an ensemble supervised machine learning algorithm that combines multiple decision trees using *Bagging (Bootstrap Aggregation)*.

VIVA SUMMARY (Hinglish):
Single Decision tree easily overfit ho jata hai (data rat leta hai). Random Forest 100 alag-alag Decision Trees banata hai. Har tree random features aur random data par train hota hai. Jab naya link/SMS aata hai, sabhi 100 trees vote karte hain: agar 85 trees 'Phishing' bolte hain, toh final verdict
85% Scam Probability
hoti hai!

- **Why Random Forest is Ideal for Phishing Detection:**
  • Heterogeneous Features: URL length, presence of '@', homoglyphs (Cyrillic characters), digit count, entropy, subdomain depth.
  • Low Variance & Overfitting Resistance: Averaging across hundreds of uncorrelated trees eliminates individual tree bias.
  • Feature Importance: Tells us which factor contributed most to the scam verdict (e.g., domain age vs urgency vocabulary).

EXAMINER QUESTION: "Why use Random Forest instead of a simple Linear Regression or single Decision Tree?"

"A single Decision Tree has high variance and easily overfits on phishing keywords. Linear Regression cannot handle non-linear relationships like URL entropy and homoglyphs. Random Forest uses bagging and feature randomness to create an ensemble of uncorrelated trees, giving high accuracy and reliable threat probability scoring."

AI FOUNDATIONS #05

Supervised vs Unsupervised Learning in Security

ATTRIBUTE	SUPERVISED LEARNING	UNSUPERVISED LEARNING
Training Data	Labeled Data (Input X + Ground Truth Label Y)	Unlabeled Data (Only Input X, No Labels)
Objective	Learn mapping function Y = f(X) for classification/prediction	Discover hidden patterns, clusters, and anomalies
Security Use Case	**Phishing & Spam Classification:** Trained on 50,000 known scam and genuine emails.	**Zero-Day & DDoS Detection:** Detecting anomalous spikes in network traffic without prior knowledge.
Key Algorithms	Random Forest, SVM, Logistic Regression, Naive Bayes	K-Means Clustering, Isolation Forest, DBSCAN, Autoencoders

EXAMINER QUESTION: "Can Supervised Learning detect a Zero-Day attack?"

"No, sir. Supervised learning requires labeled historical examples. Since a zero-day vulnerability has never been seen before, there are no training labels. That is why Unsupervised Anomaly Detection (like Isolation Forest or autoencoders) is used for zero-day defense, because it flags anything that deviates from normal baseline cluster behavior."

LLM ENGINE ARCHITECTURE #06

Chatbot API Integration & Custom System Prompting

In both **Codexa** and **CyberShield**, we integrate ultra-fast LLM engines (Groq LLaMA 3.3 70B & Gemini API).

VIVA SUMMARY (Hinglish):
Chatbot API ko integrate karte waqt 3 roles hote hain: `system`, `user`, aur `assistant`.
System Prompt
LLM ka 'Character' aur 'Rules' define karta hai. Jaise Codexa mein humne system prompt diya hai ki *'You are an expert static security auditor, output only JSON with fix'*.

- **API Request Structure:**

```
const response = await fetch("https://api.groq.com/openai/v1/chat/completions", {
  method: "POST",
  headers: { "Authorization": `Bearer ${API_KEY}`, "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "llama-3.3-70b-versatile",
    temperature: 0.2, // Low temperature for deterministic security logic
    messages: [
      { role: "system", content: "You are Codexa Engine. Analyze code for OWASP flaws. Output JSON." },
      { role: "user", content: userCodeSnippet }
    ]
  })
});
```

- **Key Hyperparameters to Mention:**
  • **Temperature (0.2):** Controls randomness. We keep it low (0.1–0.2) so security code analysis is strictly factual and reproducible.
  • **Streaming (SSE):** We stream responses via Server-Sent Events to render a typewriter effect in real-time.

BROWSER RUNTIME DEFENSE #07

DOM Parsing & Threat Inspection

The **DOM (Document Object Model)** is the tree-structured in-memory representation of an HTML document constructed by the browser rendering engine.

VIVA SUMMARY (Hinglish):
DOM Parsing matlab raw HTML text ko parse karke uske elements (tags, attributes, text) ka tree structure banana. Security mein DOM Parsing se hum malicious `<script>` tags, hidden iframes, aur phishing lookalike links detect karte hain.

- **Homoglyph Detection:** Phishing URLs use Cyrillic 'а' (U+0430) instead of Latin 'a' (U+0061) to fake `google.com`. DOM parsing reads the inner text vs href attribute to flag homoglyph masquerading.
- **DOM-based XSS Defense:** In our frontend, we use `DOMPurify.sanitize()` to strip malicious event handlers (`onerror`, `onload`) before dynamically injecting content into the DOM tree.

EXAMINER READY SCRIPT

Your 2-Minute Codexa & CyberShield Intro Speech

Memorize this exact flow when the examiner says: "Introduce your project and your contribution."

"Good morning respected external examiner. My name is Jainam.

In our project, I handled the **entire end-to-end architecture of Codexa**, as well as the **interactive defense frontend of CyberShield**.

Codexa is an intelligent developer security companion. Traditional code linters only find syntax errors, but Codexa converts source code into an **Abstract Syntax Tree (AST)** to pinpoint deep architectural vulnerabilities—such as SQL injections, unvalidated redirects, and cryptographic weaknesses according to the **OWASP Top 10** standard. We integrate the high-performance **Groq LLaMA 3.3 70B engine** with specialized system prompt engineering to generate automated, zero-shot code patches.

On **CyberShield**, I developed the complete reactive command console, the **interactive 3D Cyber Threat Globe** with real-time vector arcs, our **k-Anonymity Identity Leak Auditor** that audits credentials without compromising privacy, and the **Heuristics Link & Scam Scanner** powered by ensemble decision trees.

I would be delighted to demonstrate the live AST scanning and container simulations for you."